

AI Copilot for Context-Aware Automated Code Review Using CodeLlama, CodeBERT, Static Analysis, and Retrieval-Augmented Generation

Bandi Srihitha
Department of CSE
2420030099

[KL University, Hyderabad, India]

Emani Mahathi
Department of CSE
2420030093

[KL University, Hyderabad, India]

Prathipati Pranathi
Department of CSE
2420030043

[KL University, Hyderabad, India]

. Under the guidance of Dr. Katanguri Swanthana, Department of Computer Science and Engineering.

Abstract—Manual code review is essential for software quality, security, and maintainability, yet it is slow and depends heavily on reviewer expertise. Rule-based static analyzers report deterministic findings but lack semantic and repository-level understanding, whereas large language models (LLMs) produce fluent explanations but may hallucinate or ignore project context. This paper presents an AI Copilot for context-aware automated code review that integrates language-specific static analysis (Pylint, PMD, ESLint), CodeBERT semantic embeddings, a FAISS vector index, retrieval-augmented generation (RAG), and CodeLlama-7B-Instruct. Submitted Python, Java, or JavaScript code is analyzed, similar code chunks are retrieved from CodeSearchNet and GitHub repositories, and a structured prompt combining the code, static findings, and retrieved context is passed to CodeLlama to produce review comments with explanations, suggested fixes, severity levels, supporting evidence, and confidence scores. A working prototype with a Streamlit interface is evaluated: the static-analysis stage reaches an overall precision of 0.67, recall of 0.63, and F1-score of 0.65 across the three languages, and a Java case study illustrates evidence-backed, severity-labelled reviews together with observed weaknesses. The framework is built entirely from open-source components, runs locally with a 4-bit quantized model, and offers a modular foundation for IDE and CI/CD integration.

Index Terms—code review, large language models, CodeLlama, CodeBERT, retrieval-augmented generation, static analysis, FAISS, explainable AI

I. INTRODUCTION

Code review is a cornerstone of software quality assurance. Since Fagan's formal inspections [1], review has evolved into a lightweight, tool-supported practice that helps teams find defects, share knowledge, and keep code maintainable [2]. Nevertheless, manual review consumes substantial developer time, its quality depends on the reviewer's experience, and it scales poorly as code bases and pull-request volumes grow.

Two families of automation address this problem, each with a characteristic weakness. Static analyzers such as Pylint, PMD, and ESLint detect rule violations deterministically and cheaply, but they reason over syntax and predefined rules rather than intent, and they know nothing about how similar code is written elsewhere in the repository. Large language models (LLMs), in contrast, can explain code and propose fixes in natural language, yet recent studies report that their review quality is sensitive to code length and available context, that their comments can be verbose or unsupported, and that they may

produce false positives [3]–[5]. A reviewer that is both trustworthy and helpful therefore needs deterministic evidence and semantic, repository-aware context.

This paper proposes and prototypes an **AI Copilot for context-aware automated code review**. The system runs a language-specific static analyzer, retrieves semantically similar code using CodeBERT embeddings and a FAISS index, assembles the code, static findings, and retrieved context into a structured prompt, and asks CodeLlama-7B-Instruct to produce an explainable review. The contributions are:

- A hybrid, modular review pipeline that fuses static analysis, dense code retrieval, RAG, and a code-specialized LLM for Python, Java, and JavaScript.
- A structured prompt and output format that yields, for each finding, an explanation, a suggested fix, a severity level (Critical, High, Medium, Low), supporting static-analysis evidence, and a confidence score.
- An open-source, locally runnable implementation (4-bit quantized CodeLlama, CodeBERT, FAISS) with a Streamlit interface.
- An evaluation of the static-analysis stage per language, a Java case study, and a candid analysis of observed weaknesses that motivate future work.

The rest of the paper is organized as follows. Section II reviews related work and the research gap. Section III describes the proposed framework. Section IV presents datasets, implementation, and metrics. Section V reports results and discussion, and Section VI concludes.

II. RELATED WORK

A. LLM-Based Review and Quality Assessment

Early work applied general-purpose LLMs directly to review. Rasheed *et al.* describe a GPT-4-based agent framework that flags bugs, code smells, and risks, but report early-stage results without a released benchmark [3]. Yu *et al.* studied six LLMs on security code review and found that GPT-4 performed best when supplied with CWE context, while responses were verbose and accuracy degraded with code length and missing context [4]. Simões and Venson compared GPT-3.5 Turbo and GPT-4o with SonarQube on 1,641 Java classes from two open-source projects and observed correlation with SonarQube maintainability output, but the study is limited to Java and two projects [5].

B. Hybrid Static-Analysis and LLM Approaches

Jaoua *et al.* combine knowledge-based static analysis with a learning-based reviewer through data-augmented training, RAG, and output concatenation, reporting improved relevance and completeness with RAG especially effective; they note that static analyzers can still raise false positives [6]. İcöz and Biricik integrate symbolic reasoning and prompting with CodeT5, CodeBERT, and GraphCodeBERT on CodeXGLUE and report higher accuracy at the cost of extra reasoning stages [7].

C. Learning-Based and Agentic Reviewers

CodeReviewer pre-trains on large-scale review data and outperforms earlier methods on quality estimation, comment generation, and code refinement [8]. Pornprasit and Tantithamthavorn show that fine-tuning and prompt

engineering both improve automated review, although each needs task-specific data and careful configuration [9]. CodeAgent uses a multi-agent system with a supervisory QA-checker, at the price of higher architectural complexity and computational overhead [10].

D. Retrieval-Augmented Code Review

RARE uses a dense retriever to select relevant historical reviews that enrich the generator's input and outperforms baselines on two benchmarks, but its quality depends strongly on retrieval quality [11]. LAURA adds context-enriched retrieval and review exemplars, built from 301,256 diff-comment records across 1,807 GitHub projects, and depends on high-quality historical review data [12]. Table I summarizes these works.

TABLE I
SUMMARY OF RELATED WORK

Ref.	Method	Reported outcome	Limitation
[3]	GPT-4 agent framework	Detects bugs, smells, risks	Early results; no released benchmark
[4]	Six LLMs; CWE context	GPT-4 best with CWE context	Verbose; length/context sensitive
[5]	GPT-3.5/GPT-4o vs. SonarQube	Correlates with maintainability	Java only; two OSS projects
[6]	DAT + RAG + NCO hybrid	Better relevance, completeness	False positives; RAG may miss context
[7]	LLM + symbolic reasoning	Higher accuracy and efficiency	Extra reasoning stages; task-dependent
[8]	CodeReviewer pre-training	Beats prior methods on 3 tasks	Needs specialized large-scale data
[9]	Fine-tuning + prompting	Improved review automation	Needs task-specific data/config
[10]	Multi-agent LLM + QA-checker	Strong on style, vulnerability	High complexity and overhead
[11]	RARE (dense retrieval + LLM)	Beats baselines on 2 datasets	Depends on retrieval quality
[12]	LAURA (context + exemplars)	Higher review usefulness	Needs rich historical review data

E. Research Gap and Novelty

Across these studies five recurring challenges remain: limited repository context, generic or inconsistent suggestions, static analysis without semantic reasoning, no prioritization of issues, and limited explainability. Table II maps each gap to the design choice made in this work, and Table III contrasts the proposed system with common alternatives.

TABLE II
RESEARCH GAPS AND PROPOSED SOLUTIONS

Research gap	Proposed solution
Limited repository context	Retrieve similar code with CodeBERT and FAISS
Generic, inconsistent suggestions	Context-aware generation with CodeLlama
Static analysis lacks semantics	Fuse static findings with LLM reasoning
No issue prioritization	Label issues Critical/High/Medium/Low
Limited explainability	Reason, fix, evidence, confidence per finding

TABLE III
EXISTING APPROACHES VERSUS THE PROPOSED SYSTEM

Aspect	Existing	Proposed
Review basis	Generic LLM review	Context-aware AI review
Analysis	Static analysis only	Hybrid AI + static
Context	No repository context	CodeBERT + FAISS retrieval
Output	Generic suggestions	Explainable recommendations
Priority	Equal issue priority	Severity-labelled review

III. PROPOSED FRAMEWORK

A. Overview

Fig. 1 shows the architecture. Given a source file S , the framework (i) detects the language; (ii) runs the matching static analyzer to obtain findings F ; (iii) encodes the code with CodeBERT and retrieves the k most similar chunks R from a FAISS index built offline from CodeSearchNet and GitHub repositories; (iv) builds a structured prompt P ; (v) generates the review with CodeLlama-7B-Instruct; and (vi) parses the output into severity-labelled, evidence-backed findings.

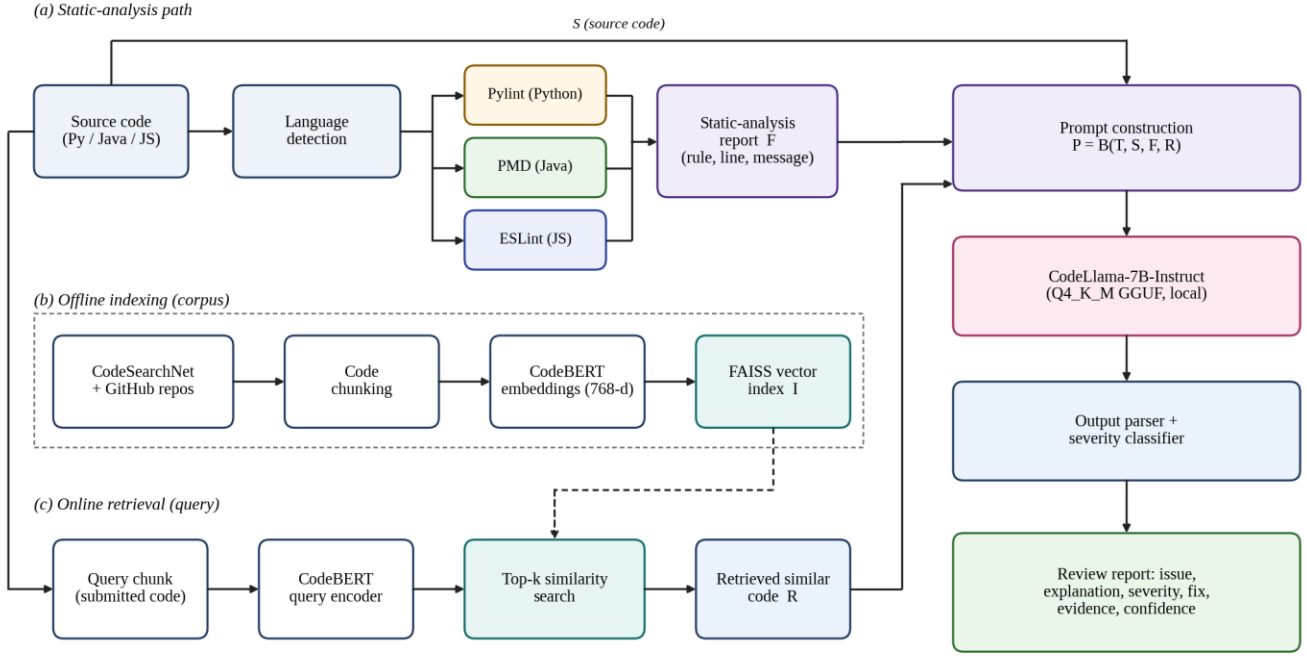


Fig. 1. Architecture of the proposed context-aware code review framework. (a) Language-specific static analysis; (b) offline construction of the FAISS index; (c) online retrieval of similar code; the results are fused into a prompt for CodeLlama.

B. Language Detection and Static Analysis

A language detector routes the input to one of three analyzers: Pylint for Python, PMD for Java, and ESLint for JavaScript [13]–[15]. Each analyzer's output is normalized into a common record containing the tool, rule identifier, line number, and message. These records form the deterministic evidence set F and later ground the LLM's claims.

C. Chunking, Embedding, and Vector Index

Source files from CodeSearchNet and selected GitHub repositories are split into code chunks. CodeBERT [16], a bimodal pre-trained model for programming and natural language, maps each chunk to a 768-dimensional vector, and the vectors are stored in a FAISS index I [17] that is built once and persisted for reuse. The indexing components (chunker, embedder, and index builder) are separate modules so that new repositories can be added without retraining any model.

D. Retrieval-Augmented Generation

At review time the submitted code is embedded as q . Similarity to an indexed chunk c_i is expressed as cosine similarity, and the top- k chunks form the retrieved context R :

$$\text{sim}(q, c_i) = (q \cdot c_i) / (\|q\| \|c_i\|) \quad (1)$$

$$R = \text{Top-}k(\{\text{sim}(q, c_i)\}, c_i \in I) \quad (2)$$

Retrieval follows the RAG paradigm [18]: rather than relying on the LLM's parametric memory alone, the model is shown concrete, similar code, which is intended to reduce generic or unsupported suggestions.

E. Structured Prompt Construction

The prompt $P = B(T, S, F, R)$ is assembled by a prompt builder from an instruction template T , the source code, the

static findings, and the retrieved context, as sketched in Listing 1. A fixed field-labelled output request makes the response machine-parsable.

```
SYSTEM: You are an expert <language> code
        reviewer. Use only the evidence given.
CODE:   <source with line numbers>
STATIC ANALYSIS (tool | rule | line | msg):
        pmd | SystemPrintln | 6 | Usage of System.out
RETRIEVED CONTEXT (top-k similar chunks):
        [1] <snippet> ... [k] <snippet>
TASK:   For each issue return: title,
        explanation, severity, suggested fix,
        evidence, confidence (0-1).
```

Listing 1. Structure of the review prompt.

F. Review Generation and Post-Processing

The generator is CodeLlama-7B-Instruct [19], used in a 4-bit Q4_K_M GGUF quantization [20] so that inference runs locally on commodity hardware without any fine-tuning. An output parser converts the response into structured findings, and a severity classifier assigns one of four levels: Critical, High, Medium, or Low. Each finding is displayed with its explanation, suggested fix, a confidence score in $[0, 1]$, and the supporting static-analysis evidence (line, message, rule, and tool). The complete procedure is given in Algorithm 1.

Algorithm 1. Context-aware review generation**Input:** source code S , FAISS index I , template T , top- k **Output:** structured review report

```

1:  $l \leftarrow \text{DetectLanguage}(S)$ 
2:  $F \leftarrow \text{StaticAnalyze}(S, l) \ \{\text{Pylint} \mid \text{PMD} \mid \text{ESLint}\}$ 
3:  $q \leftarrow \text{CodeBERT}(S)$ 
4:  $R \leftarrow \text{FAISS.Search}(I, q, k)$ 
5:  $P \leftarrow \text{BuildPrompt}(T, S, F, R)$ 
6:  $G \leftarrow \text{CodeLlama}(P)$ 
7: for each finding  $g$  in  $\text{Parse}(G)$  do
8:   assign severity, confidence, and evidence  $\subseteq F$ 
9: end for
10: return findings with per-severity summary counts

```

functions used to populate the retrieval index; CodeXGLUE [22] provides benchmark data for evaluating generated reviews; and public GitHub repositories supply repository-level context for RAG retrieval and testing.

TABLE IV
DATASETS AND THEIR ROLE IN THE WORKFLOW

Dataset	Purpose	Used in
CodeSearchNet	Similar-code corpus	CodeBERT embeddings, FAISS retrieval
CodeXGLUE	Code review benchmark	Performance evaluation
GitHub public repositories	Repository context	RAG retrieval and testing

IV. DATASETS AND EXPERIMENTAL SETUP

A. Datasets

Three sources play distinct roles, as summarized in Table IV and Fig. 2. CodeSearchNet [21] supplies a large corpus of

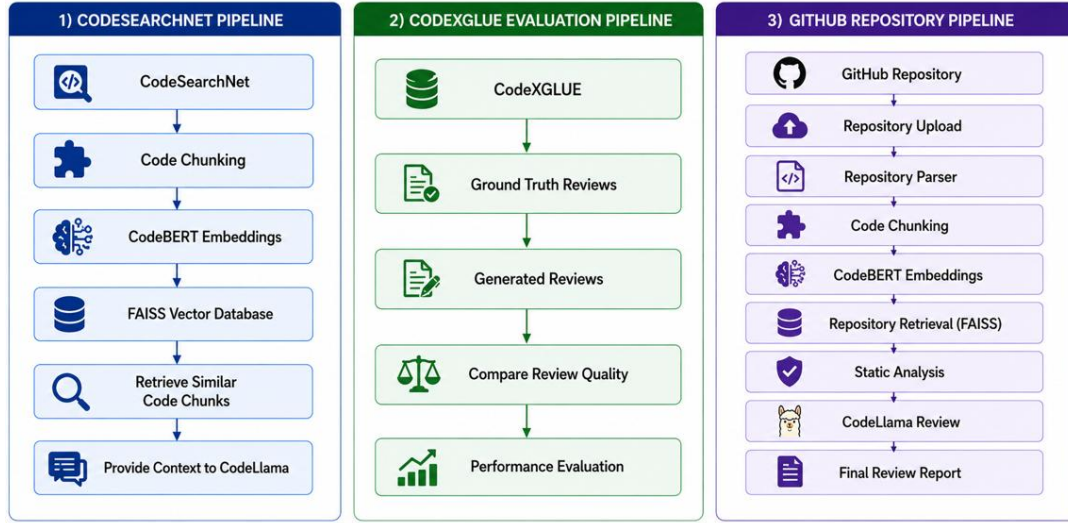


Fig. 2. How each dataset fits into the workflow: (1) CodeSearchNet feeds the retrieval index, (2) CodeXGLUE supports comparison of generated and ground-truth reviews, and (3) GitHub repositories are processed end to end.

B. Implementation

The system is implemented in Python with a modular source tree: language detector, three analyzer wrappers, chunker, CodeBERT embedder, FAISS store and retriever, prompt builder, CodeLlama client, output parser, severity classifier, and an evaluation package. The user interface is a Streamlit application [23] in which a developer pastes code and presses *Review Code*. CodeBERT (microsoft/codebert-base) runs on CPU, and CodeLlama-7B-Instruct is loaded from a local Q4_K_M GGUF file. Development followed four phases of roughly one week each: literature survey, dataset collection, chunking and FAISS setup; analyzer, CodeBERT and CodeLlama integration; RAG implementation, prompt engineering, and explainable output; and testing, evaluation, and documentation.

C. Evaluation Metrics

Detection quality is measured with precision, recall, F1-score, and accuracy computed from true/false positives and negatives (TP, FP, TN, FN):

$$P = TP / (TP + FP) \quad (3)$$

$$R = TP / (TP + FN) \quad (4)$$

$$F_1 = 2PR / (P + R) \quad (5)$$

$$\text{Acc} = (TP + TN) / (TP + TN + FP + FN) \quad (6)$$

Generated review text is to be compared against ground-truth reviews with CodeBLEU [24], which augments BLEU with syntax- and data-flow-aware matching:

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_w + \gamma \cdot \text{Match}_{\text{ast}} + \delta \cdot \text{Match}_{\text{df}} \quad (7)$$

V. RESULTS AND DISCUSSION

A. Static-Analysis Stage

The evaluation package scores the static-analysis stage separately for each language. Table V and Fig. 3 report the results of this run; the overall row is the macro-average of the three languages.

TABLE V
STATIC-ANALYSIS DETECTION RESULTS

Language	Precision	Recall	F1	Accuracy
Python	0.68	0.64	0.66	0.64
Java	0.61	0.57	0.59	0.57
JavaScript	0.72	0.68	0.70	0.68
Overall	0.67	0.63	0.65	0.63

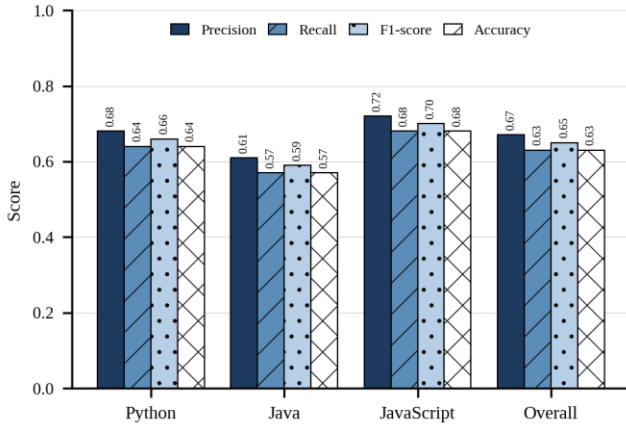


Fig. 3. Precision, recall, F1-score, and accuracy of the static-analysis stage per language.

JavaScript (ESLint) achieves the best balance ($F1 = 0.70$), Python (Pylint) is intermediate ($F1 = 0.66$), and Java (PMD) is lowest ($F1 = 0.59$), with recall trailing precision in every language. A plausible explanation is that PMD's default rule set targets style and best-practice concerns and therefore misses semantic defects, but confirming this needs a per-rule error analysis, which we leave for future work. The moderate recall also confirms the motivation for the design: rule-based findings alone leave many issues undetected, which the LLM and retrieved context are meant to complement.

B. Case Study: Java Credential Handling

To examine end-to-end behavior, the interface was given a short Java program that hard-codes a password and compares it with `==` (Listing 2). The system detected the language as Java and reported three findings (1 Critical, 2 High), shown in Fig. 4 and Table VI.

```
public class Test {
    public static void main(String[] args) {
        String password = "admin123";

        if (password == "admin123") {
            System.out.println("Login successful");
        }
    }
}
```

Listing 2. Java input used in the case study.

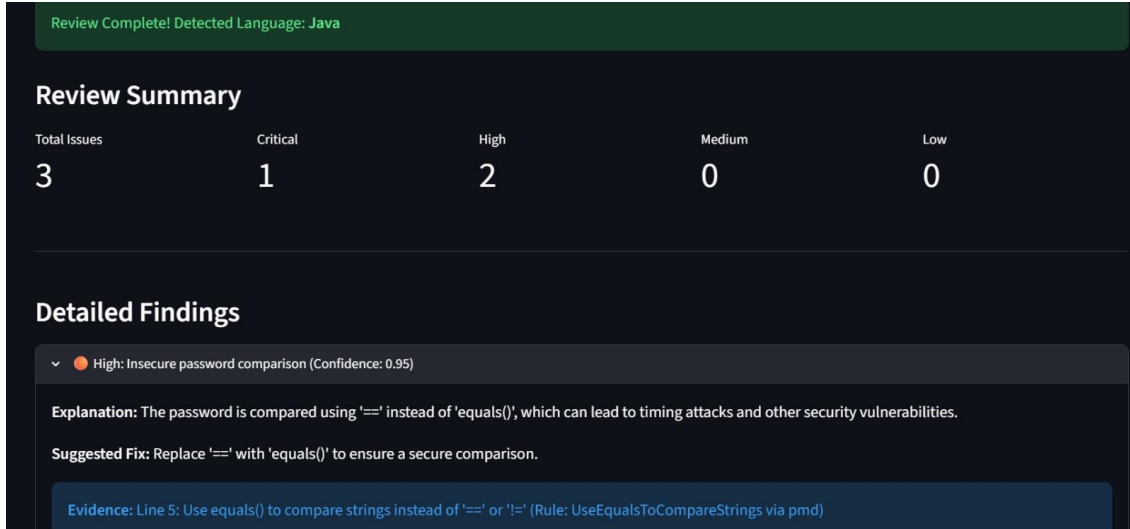


Fig. 4. Streamlit interface output for the case study: detected language, severity summary, and an expandable finding with explanation, suggested fix, and static-analysis evidence.

TABLE VI
FINDINGS REPORTED FOR THE CASE STUDY

Severity	Finding	Conf.	Static evidence (PMD)
Critical	Insecure password storage	0.95	L1, TestClassWithoutTestCases

Severity	Finding	Conf.	Static evidence (PMD)
High	Insecure password comparison	0.95	L5, UseEqualsToCompareStrings
High	Insecure system output	0.95	L6, SystemPrintln

Strengths. The LLM identified the most important defect, a plaintext hard-coded credential, which none of the PMD rules reported for this input describes, and recommended a secure hash function such as BCrypt or Argon2. For the comparison defect it proposed `equals()`, and the accompanying evidence (line 5, rule `UseEqualsToCompareStrings`) came directly from PMD, illustrating how deterministic evidence anchors a generated explanation.

Weaknesses. The output also exposes limitations that we consider important to report. (i) All three findings carry the identical confidence of 0.95, so the score does not discriminate between strong and weak findings and is not calibrated. (ii) The evidence attached to the Critical finding (`TestClassWithoutTestCases`) is unrelated to password storage, so evidence-to-finding matching needs stricter validation. (iii) The explanation of the comparison issue cites timing attacks, whereas `==` on strings compares object references rather than contents and is chiefly a correctness bug. (iv) In our judgement, flagging `System.out` usage as High severity over-ranks a minor issue. These observations are consistent with the hallucination and false-positive risks reported for LLM reviewers [4], [6], and they motivate confidence calibration and evidence verification as next steps.

C. Feasibility and Deployment

All components are open source and no model training is required, so development and deployment cost is low. The quantized 7B model and CPU-based CodeBERT make local execution possible without cloud services, which also keeps proprietary code on the developer's machine. The modular design allows additional analyzers or languages to be added by implementing one more analyzer wrapper.

D. Limitations and Threats to Validity

The reported numbers come from a single run of the prototype's evaluation harness without confidence intervals, so they should be read as indicative rather than definitive. The FAISS index used in the recorded evaluation run contained only eight vectors, so retrieval quality at CodeSearchNet scale and review-quality metrics such as CodeBLEU on CodeXGLUE and real-world repositories are not yet reported. No human study of usefulness was performed, only three languages are supported, and the 4-bit 7B model has lower capacity than larger LLMs. Static analyzers also require their runtimes (Python, Java, Node.js) to be installed.

VI. CONCLUSION AND FUTURE WORK

We presented a hybrid AI Copilot that combines static analysis, CodeBERT-based semantic retrieval, RAG, and CodeLlama to produce explainable, severity-labelled code reviews for Python, Java, and JavaScript. The prototype runs locally, exposes a simple interface, and reaches an overall static-analysis F1-score of 0.65, while the case study shows both the value of evidence-backed suggestions and concrete weaknesses in confidence estimation and evidence matching.

Future work includes (1) building the full CodeSearchNet and repository index and reporting retrieval and CodeBLEU results on CodeXGLUE; (2) confidence calibration and automatic verification that each finding's evidence supports its claim; (3) severity-sorted reports and human evaluation of usefulness; (4) ablations of static analysis and RAG; and (5) IDE and CI/CD integration and support for additional programming languages.

ACKNOWLEDGMENT

The authors thank their guide, Dr. Katanguri Swanthana, Department of Computer Science and Engineering, for guidance and encouragement throughout this project.

REFERENCES

- [1] M. E. Fagan, "Design and code inspections to reduce errors in program development," *IBM Syst. J.*, vol. 15, no. 3, pp. 182–211, 1976.
- [2] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, 2013, pp. 712–721.
- [3] Z. Rasheed *et al.*, "AI-powered code review with LLMs: Early results," arXiv:2404.18496, 2024.
- [4] J. Yu *et al.*, "An insight into security code review with LLMs: Capabilities, obstacles, and influential factors," arXiv:2401.16310, 2024.
- [5] I. R. da S. Simões and E. Venson, "Evaluating source code quality with large language models: A comparative study," in *Proc. Brazilian Symp. Softw. Quality (SBQS)*, 2024, doi: 10.1145/3701625.3701650.
- [6] I. Jaoua, O. Ben Sghaier, and H. Sahraoui, "Combining large language models with static analyzers for code review generation," in *Proc. IEEE/ACM 22nd Int. Conf. Mining Softw. Repositories (MSR)*, 2025, pp. 174–186.
- [7] B. İçöz and G. Biricik, "Automated code review using large language models with symbolic reasoning," arXiv:2507.18476, 2025.
- [8] Z. Li *et al.*, "Automating code review activities by large-scale pre-training," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2022, arXiv:2203.09095.
- [9] C. Pornprasit and C. Tantithamthavorn, "Fine-tuning and prompt engineering for large language models-based code review automation," *Inf. Softw. Technol.*, vol. 175, Art. no. 107523, Nov. 2024.
- [10] X. Tang *et al.*, "CodeAgent: Autonomous communicative agents for code review," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2024, arXiv:2402.02172.
- [11] Q. Meng, X. Zhang, Z. Ren, and J. Visser, "Code review automation using retrieval augmented generation," arXiv:2511.05302, 2025.
- [12] Y. Zhang, Y. Zhang, Z. Sun, Y. Jiang, and H. Liu, "LAURA: Enhancing code review generation with context-enriched retrieval-augmented LLM," arXiv:2512.01356, 2025.
- [13] "Pylint: Code static checker for Python." [Online]. Available: <https://pylint.readthedocs.io>
- [14] "PMD: An extensible cross-language static code analyzer." [Online]. Available: <https://pmd.github.io>
- [15] "ESLint: Pluggable JavaScript linter." [Online]. Available: <https://eslint.org>
- [16] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings Assoc. Comput. Linguistics: EMNLP*, 2020, pp. 1536–1547.
- [17] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [18] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [19] B. Rozière *et al.*, "Code Llama: Open foundation models for code," arXiv:2308.12950, 2023.
- [20] "llama.cpp: LLM inference in C/C++ (GGUF format)." [Online]. Available: <https://github.com/ggml-org/llama.cpp>

- [21] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," arXiv:1909.09436, 2019.
- [22] S. Lu *et al.*, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021, arXiv:2102.04664.
- [23] "Streamlit: A faster way to build and share data apps." [Online]. Available: <https://streamlit.io>
- [24] S. Ren *et al.*, "CodeBLEU: A method for automatic evaluation of code synthesis," arXiv:2009.10297, 2020.